Ruggine: App di Chat Testuale

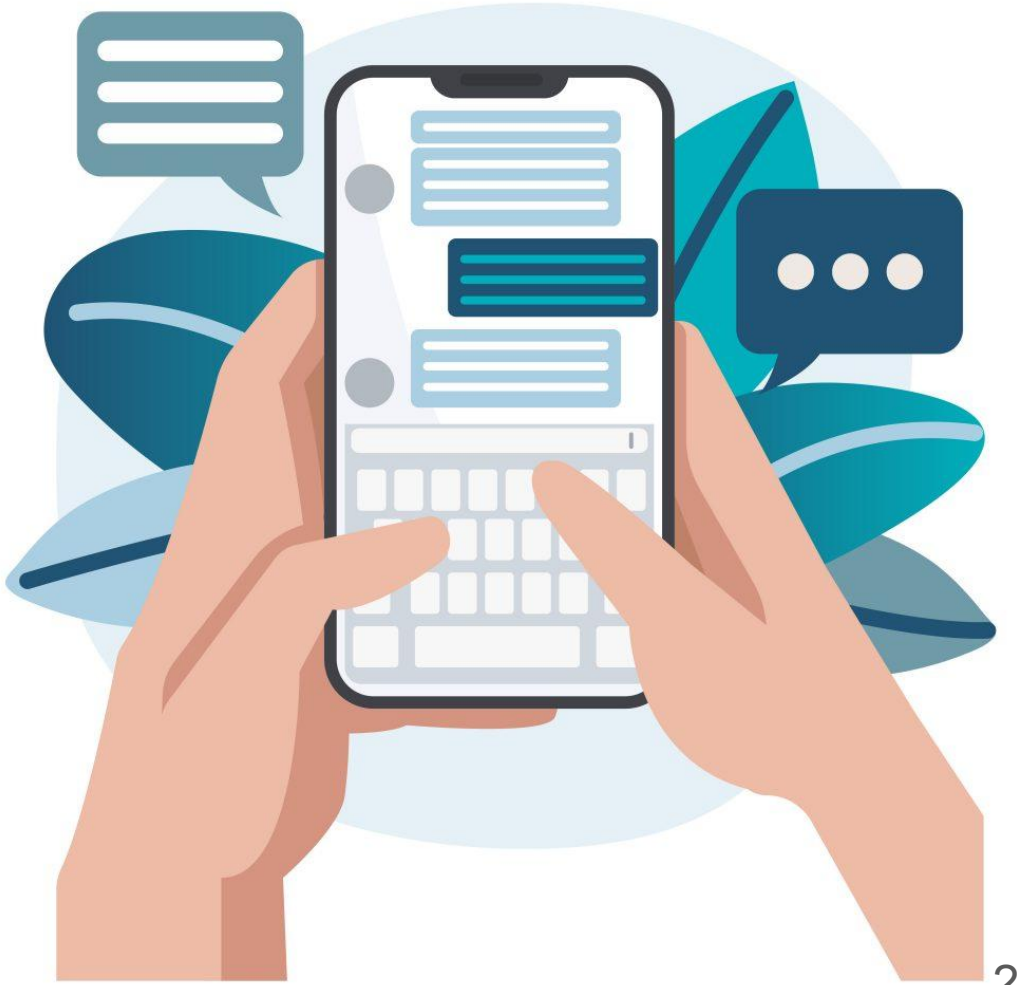
Quaglia Pietro: s349486
Sedia Tommaso: s346565
Ugliano Andrea: s346317
Vianello Samuele: s354790 (s293382)



Politecnico di Torino

Cos’è Ruggine?

- App di **messaggistica** multi-utente.
- Messaggistica **privata** e di **gruppo.**
- **Autenticazione** dell’utente.

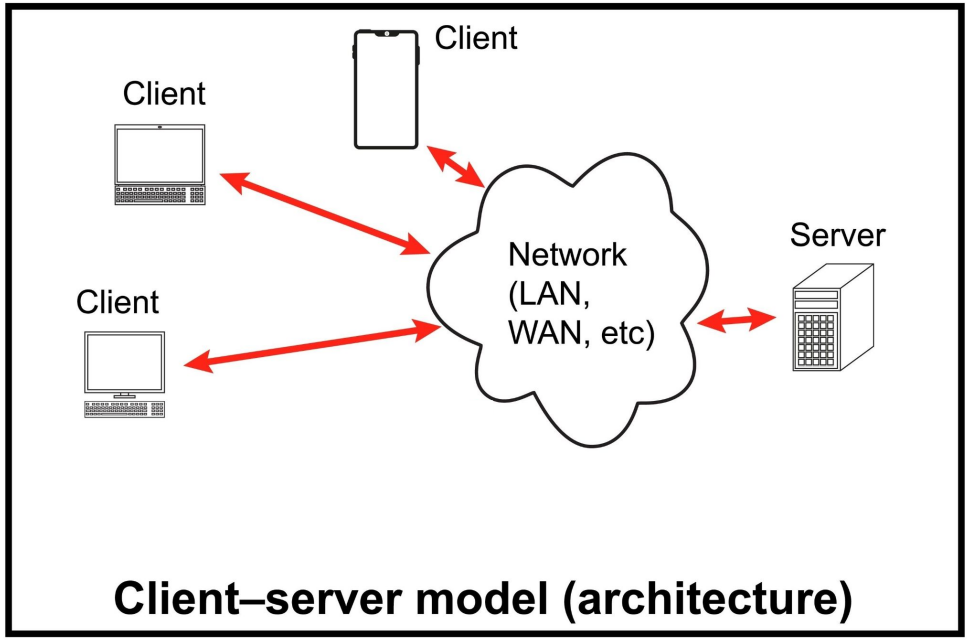


Politecnico di Torino

Architettura Ruggine

Architettura client-server.
Componenti del sistema:

- **Core Backend** (`**server.rs**`) : Gestisce la connessione dei client e i messaggi inviati.
- **Client CLI** (`**client.rs**`) : Interfaccia da terminale per l'utilizzo di ruggine.
- **Client GUI** (`**gui_client.rs**`) : Interfaccia grafica con salvataggi locali.



Full persistance



Contatti personali salvati



Gruppi in cui si è entrati



Inviti a gruppi ricevuti



Credenziali di accesso

Salvataggio Dati

```rust
pub async fn save_data<T: Serialize>(data: &T, path: &str) {
    match serde_json::to_string_pretty(data) {
        Ok(json: String) => match tokio::fs::write(path, contents: json).await {
            Ok(_) => println!("Dati salvati su disco."),
            Err(err: Error) => eprintln!("Errore nel salvataggio dei dati su disco: {}", err),
        },
        Err(err: Error) => eprintln!("Errore nella serializzazione dei dati: {}", err),
    }
}
```

Dati salvati in un file **JSON** interno al server utilizzati per memorizzare informazioni sulle sessioni degli utenti **tramite la libreria** di RUST **serde**.

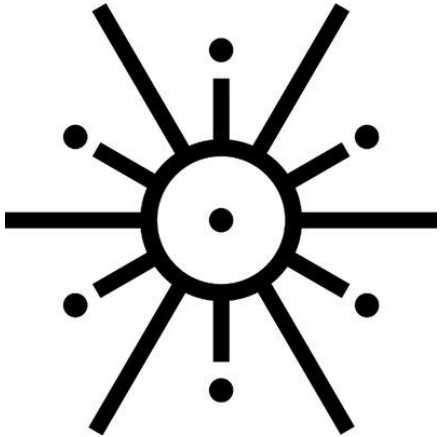
Error handling

- **Type-Safety (Nessuna Eccezione):** Utilizzo nativo dei tipi ***Result<T, E>*** e ***Option<T>*** di Rust per forzare la **gestione degli errori** a tempo di compilazione.
- **Pattern Matching:** Gestione esplicita e sicura dei percorsi di successo *(**Ok**)* e fallimento *(**Err**)* per le **operazioni di I/O**.
- **Protocollo Strutturato:** Creazione del **pacchetto** dedicato **MessageType::Error** per instradare i fallimenti logici (es. password errata, utente inesistente) dal server alla UI.
- **Graceful Shutdown:** Intercettazione delle **disconnessioni TCP con** chiusura controllata dei socket, **salvataggio dello stato** e chiusura ritardata della GUI per mostrare l'errore all'utente.



Gestione della Concorrenza - Tokio

- **Runtime Asincrono: Integrazione di Tokio** per l'utilizzo della sintassi **async/await** di Rust.
- **Multi-Thread:** Multiplexing di **molteplici** connessioni simultanee tramite **thread**.
- **I/O Non Bloccante: Disaccoppiamento** della ricezione dei **pacchetti** di rete dal rendering dell'**interfaccia** grafica per prevenire blocchi (freeze).
- **Lock in Lettura/Scrittura** : **Lettura e Scrittura** sul file JSON **senza conflitti** con altre operazioni simili.



Architettura del server

- **Memoria Condivisa e Sicura** *(**Arc<Mutex<T>>**)*: Proprietà condivisa tra tutti i task TCP con accesso in scrittura mutuamente esclusivo, prevenendo le **data race**
- **Gestione delle Connessioni:**
  - **Task Leggeri** *(tokio::spawn)***:** Un green-thread **asincrono** e indipendente per ogni nuovo utente, per non bloccare il sistema
  - **Split del Socket:** La singola connessione TCP viene divisa in due flussi isolati un **reader** e un **writer** tramite *.into_split()*
- **Disaccoppiamento I/O (Canali MPSC):** Il server deposita i messaggi in una coda asincrona sbloccando la memoria, e delega l'invio fisico sulla rete per evitare colli di bottiglia.



Autenticazione e Sicurezza

- **Nessuna password salvata in chiaro**: il server genera un hash esadecimale tramite l'algoritmo **SHA-256**.
- Registrazione **trasparente**: il primo accesso con un nuovo username registra automaticamente l'utente e salva l'hash della password nel file json.
- Controllo in tempo reale per impedire "**doppi accessi**" con lo stesso account.



Client



- Creato per **leggerezza** ed esecuzione **headless**.
- **Compatibile** con i principali sistemi (windows, android, linux…) dopo la compilazione.
- Sfrutta due task **asincroni** paralleli:
- **Task 1**: Attende input da tastiera (`stdin`), mappa i comandi testuali (es. `/msg`) in `MessageType` e invia il JSON.
- **Task 2**: Ascolta il TCP in background e stampa a video i messaggi colorati e formattati.

GUI

La versione con GUI del client usa un modello a **due thread** per gestire separatamente la **UI** e le **comunicazioni** con il **server**

- **eframe/egui (GUI)** → richiede il controllo totale del thread principale, è bloccante per natura — eframe::run_native non ritorna mai. Usa un semplice std::sync::mpsc per ricevere messaggi (nessun async necessario)
- **Runtime asincrono Tokio** → necessita di un proprio runtime quindi non può condividere il thread principale. Vive su un thread OS dedicato (thread::spawn) ed usa un tokio::sync::mpsc per comunicare (ricezione awaitable, non bloccante). Inoltre avvia un sottoprocesso async leggero per il logging della CPU

Il thread di rete deve gestire **due** sorgenti di **eventi contemporaneamente**:

- i **messaggi in arrivo** dal server TCP
- i **comandi inviati** dalla GUI

per farlo, **tokio::select!** mette il **task in attesa** su entrambi i canali allo stesso tempo. Quando uno dei due produce un **evento**, il **task si risveglia** ed esegue il ramo corrispondente.

GUI: Startup sequence

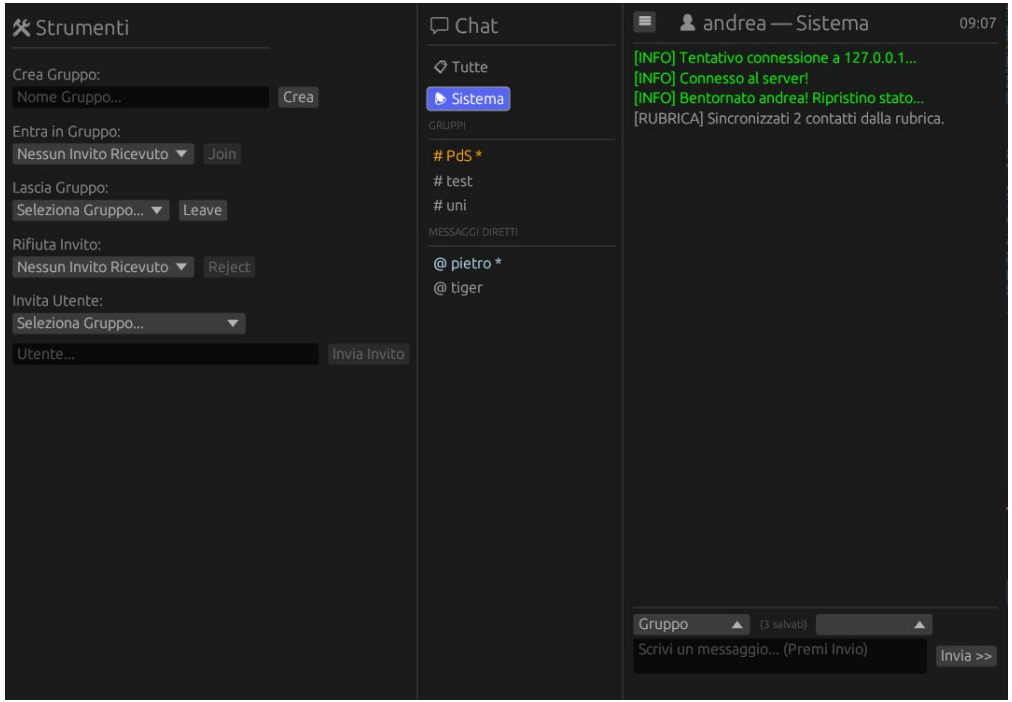
1. Due **canali mpsc** vengono creati prima che alcun thread esista:
   a. **tokio** → **asincrono** per gestire i messaggi in arrivo dal server.
   b. **gui** → **sincrono**, per gestire i comandi inviati dalla gui.
2. thread::spawn crea un nuovo **thread OS** al cui interno viene costruito un runtime Tokio completo e block_on guida **run_network** che rimane in attesa di una connessione al server **bloccando il thread principale** e successivamente **avviando il loop degli eventi con entrambi i canali** a disposizione.
3. Quando al GUI invia un **messaggio Start** in run_network tenta **TCPStream::connect** che solo una volta che ha avuto successo prosegue col codice.
4. tokio::spawn avvia il **logger della CPU** → il loop principale con **tokio::select! gestisce** contemporanemente i **messaggi del server** e i **comandi della GUI**.

GUI: Gestione messaggi e Aggiornamento GUI

Il thread della GUI chiama **try_recv()** in modo non bloccante ad ogni frame per svuotare tutti i messaggi in arrivo dal thread principale che sposta sul canale della GUI i messaggi che gli arrivano dal server attraverso la connessione Tcp.

Il thread GUI poi gestice i messagggi ricevuti per

- aggiornare le variabili locali
- visualizzare i messaggi ricevuti
- aggiornare lo stato della GUI



Log

Il **server** fa logging di:

- canale di ascolto e **nuove connessioni**
- **login/logout** di utenti
- **creazione** di gruppi e inviti
- **aggiunta/rimozione** da gruppi

```
Server in ascolto su 0.0.0.0:8190
Nuova connessione: 127.0.0.1:59405
Utente Login: andrea
Nuova connessione: 127.0.0.1:59413
[SAVE] Salvataggio nuova registrazione utente: pietro
Dati salvati su disco.
Utente Login: pietro
[SAVE] Salvataggio aggiornamento contatti tra 'andrea' e 'pietro'
Dati salvati su disco.
[SAVE] Salvataggio creazione gruppo: Uni
Dati salvati su disco.
[SAVE] Salvataggio invito: utente 'andrea' nel gruppo 'Uni'
Dati salvati su disco.
[SAVE] Salvataggio ingresso nel gruppo: utente 'andrea' in 'Uni'
Dati salvati su disco.
[SAVE] Salvataggio uscita dal gruppo: utente 'andrea' da 'Uni'
Dati salvati su disco.
Disconnesso: andrea
```



Monitoraggio del sistema

- Un **sistema di logging** è innestato in background su tutti gli eseguibili.
- Sfrutta il crate `sysinfo` per catturare **PID** e percentuale d'**uso della CPU** calcolata sul numero di core.
- Ogni task si risveglia **periodicamente** (120s) appendendo i dati in un file di dump per **analisi sulle performance**.
- gestito in modo **asincrono** con Tokio





DEMO TIME





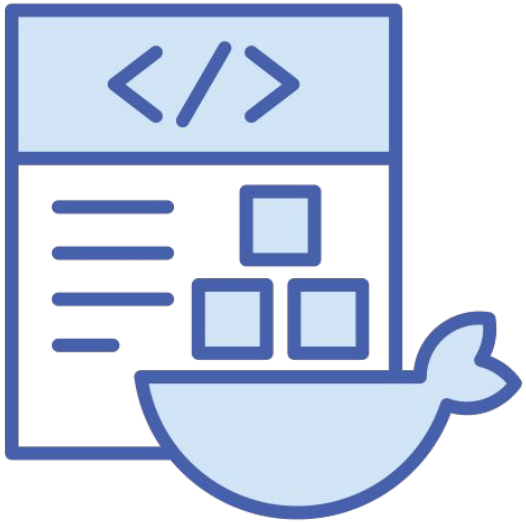
Docker

Deployment e Produzione

- Per creare un ambiente simil-produzione, viene fatto uso di un **container docker** per il server.rs
- Rende **disponibile** il server.rs **per ogni dispositivo** connesso alla subnet (implementabile in futuro con un'interfaccia web)

Ottimizzazione e Sicurezza Build

- La build veloce **consente di** spostare, riavviare e semplicemente **gestire velocemente** il server.rs
- Patch future possono essere gestite in automatico aggiornando il container su Docker Hub



Grazie per l’attenzione
^_^



Politecnico
di Torino